

```
In [1]: # This Python 3 environment comes with many helpful analytics libraries
        installed
        # It is defined by the kaggle/python Docker image:
        https://github.com/kaggle/docker-python
        # For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter)
# will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
# that gets preserved as output when you create a version using "Save &
# Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be
# saved outside of the current session

# Use the kagglehub client library to attach Kaggle resources like
# competitions, datasets, and models to your session
# Learn more about kagglehub:
# https://github.com/Kaggle/kagglehub/blob/main/README.md

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')

/kaggle/input/datasets/dptel22/j1v-c2/fire_archive_J1V-
C2_794139.csv
/kaggle/input/datasets/dptel22/j1v-c2/fire_nrt_J1V-C2_794139.csv
/kaggle/input/datasets/dptel22/sv-c2snpp/fire_nrt_SV-C2_794140.csv
/kaggle/input/datasets/dptel22/sv-c2snpp/fire_archive_SV-
C2_794140.csv
```

Import libraries

```
In [2]: !pip install pandas numpy scipy statsmodels  
!pip install matplotlib seaborn plotly  
!pip install geopandas h3 shapely
```

```
Requirement already satisfied: pandas in  
/usr/local/lib/python3.12/dist-packages (2.3.3)  
Requirement already satisfied: numpy in  
/usr/local/lib/python3.12/dist-packages (2.0.2)  
Requirement already satisfied: scipy in  
/usr/local/lib/python3.12/dist-packages (1.16.3)  
Requirement already satisfied: statsmodels in  
/usr/local/lib/python3.12/dist-packages (0.14.6)  
Requirement already satisfied: python-dateutil>=2.8.2 in  
/usr/local/lib/python3.12/dist-packages (from pandas)  
(2.9.0.post0)  
Requirement already satisfied: pytz>=2020.1 in  
/usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)  
Requirement already satisfied: tzdata>=2022.7 in  
/usr/local/lib/python3.12/dist-packages (from pandas) (2026.1)  
Requirement already satisfied: patsy>=0.5.6 in  
/usr/local/lib/python3.12/dist-packages (from statsmodels) (1.0.2)  
Requirement already satisfied: packaging>=21.3 in  
/usr/local/lib/python3.12/dist-packages (from statsmodels) (26.1)  
Requirement already satisfied: six>=1.5 in  
/usr/local/lib/python3.12/dist-packages (from python-  
dateutil>=2.8.2->pandas) (1.17.0)  
Requirement already satisfied: matplotlib in  
/usr/local/lib/python3.12/dist-packages (3.10.0)  
Requirement already satisfied: seaborn in  
/usr/local/lib/python3.12/dist-packages (0.13.2)  
Requirement already satisfied: plotly in  
/usr/local/lib/python3.12/dist-packages (5.24.1)  
Requirement already satisfied: contourpy>=1.0.1 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)  
Requirement already satisfied: cyclor>=0.10 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)  
Requirement already satisfied: fonttools>=4.22.0 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)  
Requirement already satisfied: kiwisolver>=1.3.1 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)  
Requirement already satisfied: numpy>=1.23 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)  
Requirement already satisfied: packaging>=20.0 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.1)  
Requirement already satisfied: pillow>=8 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)  
Requirement already satisfied: pyparsing>=2.3.1 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)  
Requirement already satisfied: python-dateutil>=2.7 in  
/usr/local/lib/python3.12/dist-packages (from matplotlib)
```

```

(2.9.0.post0)
Requirement already satisfied: pandas>=1.2 in
/usr/local/lib/python3.12/dist-packages (from seaborn) (2.3.3)
Requirement already satisfied: tenacity>=6.2.0 in
/usr/local/lib/python3.12/dist-packages (from plotly) (9.1.4)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas>=1.2-
>seaborn) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas>=1.2-
>seaborn) (2026.1)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-
dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: geopandas in
/usr/local/lib/python3.12/dist-packages (1.1.3)
Collecting h3
  Downloading h3-4.5.0-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.w
hl.metadata (17 kB)
Requirement already satisfied: shapely in
/usr/local/lib/python3.12/dist-packages (2.1.2)
Requirement already satisfied: numpy>=1.24 in
/usr/local/lib/python3.12/dist-packages (from geopandas) (2.0.2)
Requirement already satisfied: pyogrio>=0.7.2 in
/usr/local/lib/python3.12/dist-packages (from geopandas) (0.12.1)
Requirement already satisfied: packaging in
/usr/local/lib/python3.12/dist-packages (from geopandas) (26.1)
Requirement already satisfied: pandas>=2.0.0 in
/usr/local/lib/python3.12/dist-packages (from geopandas) (2.3.3)
Requirement already satisfied: pyproj>=3.5.0 in
/usr/local/lib/python3.12/dist-packages (from geopandas) (3.7.2)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0-
>geopandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in
/usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0-
>geopandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in
/usr/local/lib/python3.12/dist-packages (from pandas>=2.0.0-
>geopandas) (2026.1)
Requirement already satisfied: certifi in
/usr/local/lib/python3.12/dist-packages (from pyogrio>=0.7.2-
>geopandas) (2026.4.22)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-
dateutil>=2.8.2->pandas>=2.0.0->geopandas) (1.17.0)
Downloading h3-4.5.0-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.w
hl (1.1 MB)
[2K] [90m-----[0m
[32m1.1/1.1 MB[0m [31m17.9 MB/s[0m eta
[36m0:00:00[0m[0m00:01[0m
[?25hInstalling collected packages: h3
Successfully installed h3-4.5.0

```

```
In [3]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import h3
import warnings

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)

SATURATION_TEMP = 367.0
SATURATION_BUFFER = 0.1
H3_RES = 8
```

```
In [4]: paths = {
    "j1_archive": "/kaggle/input/datasets/dptel22/j1v-
c2/fire_archive_J1V-C2_794139.csv",
    "j1_nrt": "/kaggle/input/datasets/dptel22/j1v-c2/fire_nrt_J1V-
C2_794139.csv",
    "snpp_archive": "/kaggle/input/datasets/dptel22/sv-
c2snpp/fire_archive_SV-C2_794140.csv",
    "snpp_nrt": "/kaggle/input/datasets/dptel22/sv-c2snpp/fire_nrt_SV-
C2_794140.csv",
}

OUTPUT_PATH = "/kaggle/working/sih2026_h3_daily_features.parquet"

print("Configuration loaded.")
print("H3 resolution:", H3_RES)
print("Saturation threshold:", SATURATION_TEMP, "K")
```

```
Configuration loaded.
H3 resolution: 8
Saturation threshold: 367.0 K
```

Schema Verifcation

```
In [5]: for name, path in paths.items():
        check = pd.read_csv(path, nrows=3)
        print(f"\n{name}")
        print("  path:", path)
        print("  columns:", check.columns.tolist())
        print("  satellite sample:",
check["satellite"].dropna().unique().tolist())
        print("  acq_time dtype:", check["acq_time"].dtype)

print("\nExpected schema pattern:")
print("  Archive: brightness, bright_t31, ..., type")
print("  NRT:      brightness, bright_t31, ... (no type)")
```

```
j1_archive
  path: /kaggle/input/datasets/dptel22/j1v-c2/fire_archive_J1V-
C2_794139.csv
  columns: ['latitude', 'longitude', 'brightness', 'scan',
'track', 'acq_date', 'acq_time', 'satellite', 'instrument',
'confidence', 'version', 'bright_t31', 'frp', 'daynight', 'type']
  satellite sample: ['N20']
  acq_time dtype: int64
```

```
j1_nrt
  path: /kaggle/input/datasets/dptel22/j1v-c2/fire_nrt_J1V-
C2_794139.csv
  columns: ['latitude', 'longitude', 'brightness', 'scan',
'track', 'acq_date', 'acq_time', 'satellite', 'instrument',
'confidence', 'version', 'bright_t31', 'frp', 'daynight']
  satellite sample: ['N20']
  acq_time dtype: int64
```

```
snpp_archive
  path: /kaggle/input/datasets/dptel22/sv-c2snpp/fire_archive_SV-
C2_794140.csv
  columns: ['latitude', 'longitude', 'brightness', 'scan',
'track', 'acq_date', 'acq_time', 'satellite', 'instrument',
'confidence', 'version', 'bright_t31', 'frp', 'daynight', 'type']
  satellite sample: ['SNPP']
  acq_time dtype: int64
```

```
snpp_nrt
  path: /kaggle/input/datasets/dptel22/sv-c2snpp/fire_nrt_SV-
C2_794140.csv
  columns: ['latitude', 'longitude', 'brightness', 'scan',
'track', 'acq_date', 'acq_time', 'satellite', 'instrument',
'confidence', 'version', 'bright_t31', 'frp', 'daynight']
  satellite sample: ['SNPP']
  acq_time dtype: int64
```

Expected schema pattern:

Archive: brightness, bright_t31, ..., type

NRT: brightness, bright_t31, ... (no type)

Ingestion and schema harmonization

```
In [6]: dfs = []

for name, path in paths.items():
    df_part = pd.read_csv(path)

    # Preserve provenance.
    df_part["source"] = "archive" if "archive" in name else "nrt"
    df_part["satellite_name"] = "NOAA-20" if "j1" in name else "SUOMI-
NPP"

    dfs.append(df_part)

df_raw = pd.concat(dfs, ignore_index=True)

print(f"Total raw rows (archive + NRT, before overlap removal):
{len(df_raw):,}")
print("\nRaw satellite values:")
print(df_raw["satellite"].value_counts(dropna=False))

def harmonize_firms_schema(df):
    df = df.copy()

    # Archive -> common VIIRS I-band naming.
    rename_map = {
        "brightness": "bright_ti4",
        "bright_t31": "bright_ti5",
        "type": "fire_type",
    }
    df = df.rename(columns={k: v for k, v in rename_map.items() if k in
df.columns})

    # Keep already-standardized values unchanged.
    if "confidence" in df.columns:
        df["confidence"] = df["confidence"].replace({
            "l": "low",
            "n": "nominal",
            "h": "high",
        })

    if "daynight" in df.columns:
        df["daynight"] = df["daynight"].replace({
            "D": "Day",
            "N": "Night",
        })

    # NRT rows have NaN in fire_type after concat because the NRT source
    # does not contain the original archive-only type field.
    if "fire_type" in df.columns:
        is_na = df["fire_type"].isna()
```

```

df["is_static_land"] = np.select(
    [is_na],
    [-1],
    default=(df["fire_type"] == 2).astype(int),
)

df["is_offshore"] = np.select(
    [is_na],
    [-1],
    default=(df["fire_type"] == 3).astype(int),
)
else:
    df["is_static_land"] = -1
    df["is_offshore"] = -1

return df

df = harmonize_firms_schema(df_raw)

print(f"Total rows after harmonization: {len(df):,}")
print("Harmonized columns:")
print(df.columns.tolist())

```

Total raw rows (archive + NRT, before overlap removal): 2,597,596

Raw satellite values:

satellite

N20 1329378

SNPP 1268218

Name: count, dtype: int64

Total rows after harmonization: 2,597,596

Harmonized columns:

```

['latitude', 'longitude', 'bright_ti4', 'scan', 'track',
'acq_date', 'acq_time', 'satellite', 'instrument', 'confidence',
'version', 'bright_ti5', 'frp', 'daynight', 'fire_type', 'source',
'satellite_name', 'is_static_land', 'is_offshore']

```

Tabular EDA

```
In [7]: print(f>Date range: {df['acq_date'].min()} to {df['acq_date'].max()}")
print(f"Memory usage: {df.memory_usage(deep=True).sum() / 1e6:.2f} MB")

print("\n--- Missing values (%) ---")
print((df.isnull().mean() * 100).round(2))

print(f"\nExact duplicate rows: {df.duplicated().sum():,}")

numeric_cols = ["bright_ti4", "bright_ti5", "frp", "scan", "track"]
summary = df[numeric_cols].describe(
    percentiles=[0.01, 0.05, 0.25, 0.50, 0.75, 0.95, 0.99]
).T
summary["skew"] = df[numeric_cols].skew()
summary["kurtosis"] = df[numeric_cols].kurtosis()
display(summary)

saturated_mask = df["bright_ti4"] >= (SATURATION_TEMP -
SATURATION_BUFFER)

print(
    f"\nSaturated I4 records (>= {SATURATION_TEMP - SATURATION_BUFFER}
K): "
    f"{saturated_mask.sum():,} ({saturated_mask.mean() * 100:.2f}%"
)

print("\nSaturation by satellite:")
print(df.loc[saturated_mask, "satellite_name"].value_counts())

print("\nConfidence distribution (%):")
print((df["confidence"].value_counts(normalize=True) * 100).round(2))

print("\nDay/night distribution (%):")
print((df["daynight"].value_counts(normalize=True) * 100).round(2))

print("\nFire type distribution (archive values + NRT NaN):")
print(df["fire_type"].value_counts(dropna=False))
```

```
Date range: 2024-08-01 to 2026-08-01
Memory usage: 1326.00 MB
```

```
--- Missing values (%) ---
latitude      0.00
longitude     0.00
bright_ti4    0.00
scan          0.00
track         0.00
acq_date      0.00
acq_time      0.00
satellite     0.00
instrument     0.00
```

```
confidence      0.00
version         0.00
bright_ti5      0.00
frp             0.00
daynight        0.00
fire_type       4.65
source          0.00
satellite_name  0.00
is_static_land  0.00
is_offshore     0.00
dtype: float64
```

Exact duplicate rows: 0

	count	mean	std	min	1%	5
bright_ti4	2597596.0	331.403556	16.937914	207.93	297.34	301.
bright_ti5	2597596.0	301.597048	9.746338	243.18	279.24	286.
frp	2597596.0	6.001282	13.162558	0.00	0.43	0.75
scan	2597596.0	0.452376	0.081331	0.32	0.32	0.35
track	2597596.0	0.469152	0.109133	0.36	0.36	0.36

Saturated I4 records (≥ 366.9 K): 78,309 (3.01%)

Saturation by satellite:
satellite_name
SUOMI-NPP 39909
NOAA-20 38400
Name: count, dtype: int64

Confidence distribution (%):
confidence
nominal 79.63
low 17.83
high 2.54
Name: proportion, dtype: float64

Day/night distribution (%):
daynight
Day 70.61
Night 29.39
Name: proportion, dtype: float64

Fire type distribution (archive values + NRT NaN):
fire_type
0.0 2174507
2.0 298418
NaN 120735
3.0 3912
1.0 24
Name: count, dtype: int64

Cleaning, timestamp construction, and deduplicatio

```

In [8]: # Parse acquisition date.
df["acq_date"] = pd.to_datetime(df["acq_date"], errors="raise")

# Robust acquisition-time validation.
acq_time_num = pd.to_numeric(df["acq_time"], errors="coerce")

assert acq_time_num.notna().all(), "acq_time contains non-numeric values."
assert acq_time_num.between(0, 2359).all(), "acq_time contains values outside 0000-2359."
assert (acq_time_num % 100 < 60).all(), "acq_time contains invalid minute values."

df["acq_time_str"] = (
    acq_time_num
    .round()
    .astype("int64")
    .astype(str)
    .str.zfill(4)
)

df["timestamp"] = pd.to_datetime(
    df["acq_date"].dt.strftime("%Y-%m-%d")
    + " "
    + df["acq_time_str"].str[:2]
    + ":"
    + df["acq_time_str"].str[2:],
    errors="raise",
)

# Remove archive/NRT overlap.
dedup_cols = [
    "latitude",
    "longitude",
    "acq_date",
    "acq_time",
    "satellite",
]

n_before = len(df)
df = df.drop_duplicates(subset=dedup_cols, keep="first").copy()

print(f"Dropped {n_before - len(df):,} archive/NRT overlap rows.")

# Physical sanity filters.
n_before = len(df)
df = df[df["bright_ti4"] > 200].copy()
df = df[df["frp"] >= 0].copy()

print(f"Dropped {n_before - len(df):,} rows via physical sanity filters.")

# IMPORTANT:
# .replace() preserves values that are already standardized.
# - N      -> SNPP
# - N20    -> NOAA20
# - SNPP   -> remains SNPP

```

```
df["satellite"] = df["satellite"].replace({
    "N": "SNPP",
    "N20": "NOAA20",
})

print(f"Rows after dedup + cleaning: {len(df):,}")
print("\nSatellite values after harmonization:")
print(df["satellite"].value_counts(dropna=False))
```

Dropped 2 archive/NRT overlap rows.
Dropped 0 rows via physical sanity filters.
Rows after dedup + cleaning: 2,597,594

Satellite values after harmonization:

satellite	
NOAA20	1329377
SNPP	1268217

Name: count, dtype: int64

Visual EDA

```
In [9]: fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle("FIRMS VIIRS Physical Distributions", fontsize=16)

sns.histplot(
    df["bright_ti4"],
    bins=100,
    kde=True,
    ax=axes[0, 0],
)
axes[0, 0].axvline(
    SATURATION_TEMP,
    linestyle="--",
    label=f"Saturation ({SATURATION_TEMP} K)",
)
axes[0, 0].set_title("Brightness Temperature (TI4)")
axes[0, 0].legend()

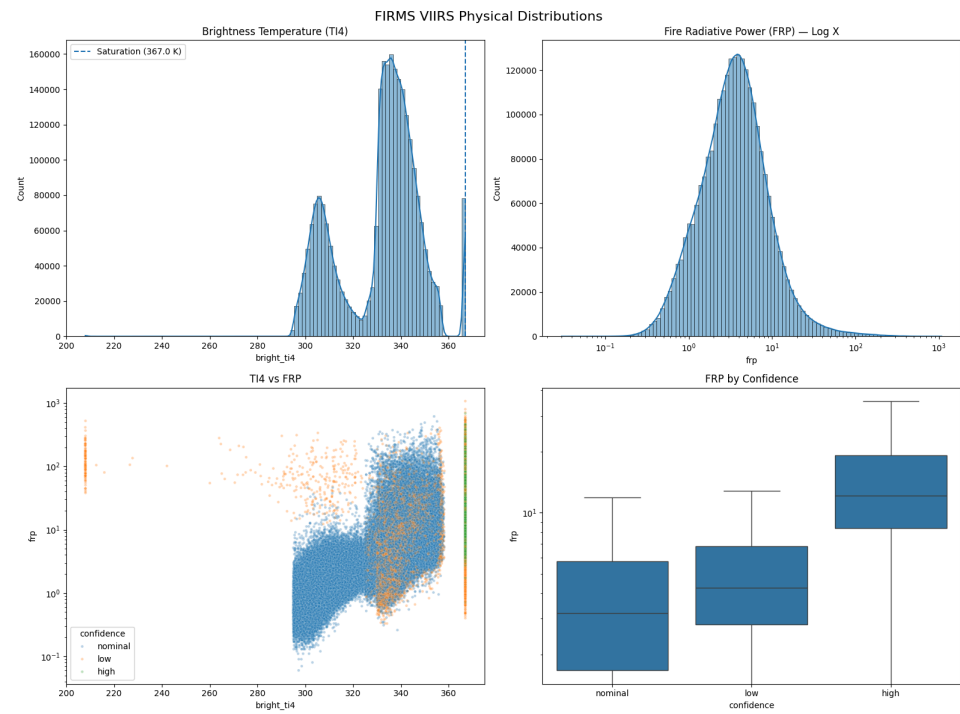
sns.histplot(
    df.loc[df["frp"] > 0, "frp"],
    bins=100,
    kde=True,
    ax=axes[0, 1],
    log_scale=(True, False),
)
axes[0, 1].set_title("Fire Radiative Power (FRP) – Log X")

sample = df.sample(frac=0.20, random_state=42)

sns.scatterplot(
    data=sample,
    x="bright_ti4",
    y="frp",
    hue="confidence",
    alpha=0.30,
    s=10,
    ax=axes[1, 0],
)
axes[1, 0].set_yscale("log")
axes[1, 0].set_title("TI4 vs FRP")

sns.boxplot(
    x="confidence",
    y="frp",
    data=df,
    ax=axes[1, 1],
    showfliers=False,
)
axes[1, 1].set_yscale("log")
axes[1, 1].set_title("FRP by Confidence")
```

```
plt.tight_layout()  
plt.show()
```



Temporal EDA

```

In [10]: # FIRMS acquisition time is UTC.
# Convert to IST for India-focused diurnal analysis.
df["timestamp_ist"] = df["timestamp"] + pd.Timedelta(hours=5,
minutes=30)
df["hour_ist"] = df["timestamp_ist"].dt.hour
df["date_only"] = df["acq_date"].dt.date

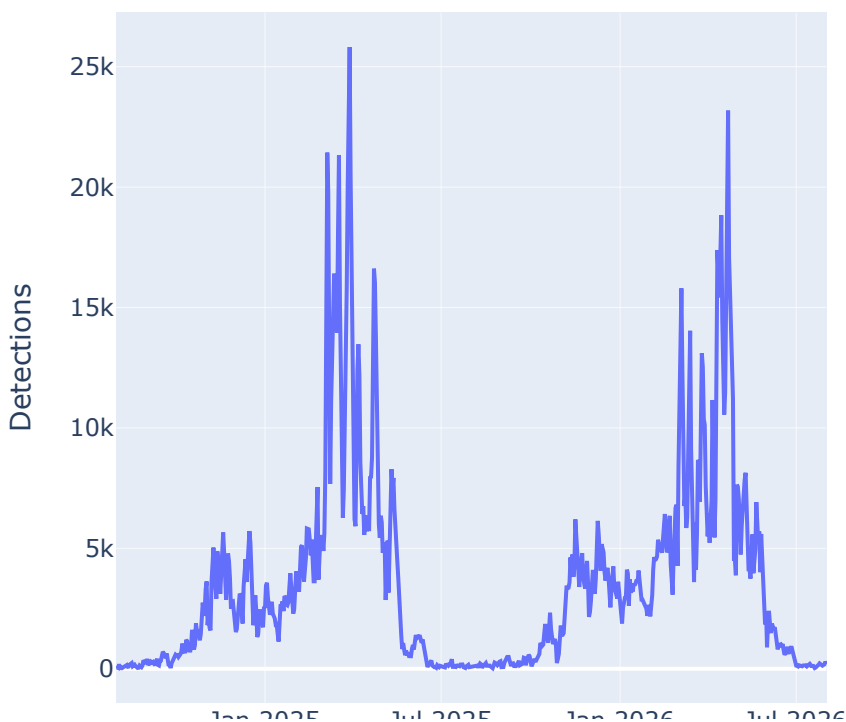
daily_counts = (
    df.groupby("date_only")
      .size()
      .reset_index(name="count")
)

fig_daily = px.line(
    daily_counts,
    x="date_only",
    y="count",
    title="Daily Thermal Anomaly Counts",
    labels={"date_only": "Date", "count": "Detections"},
)
fig_daily.show()

fig_hourly = px.histogram(
    df,
    x="hour_ist",
    color="daynight",
    barmode="overlay",
    title="Satellite Pass Times (IST)",
    labels={"hour_ist": "Hour (IST)", "count": "Detections"},
)
fig_hourly.show()

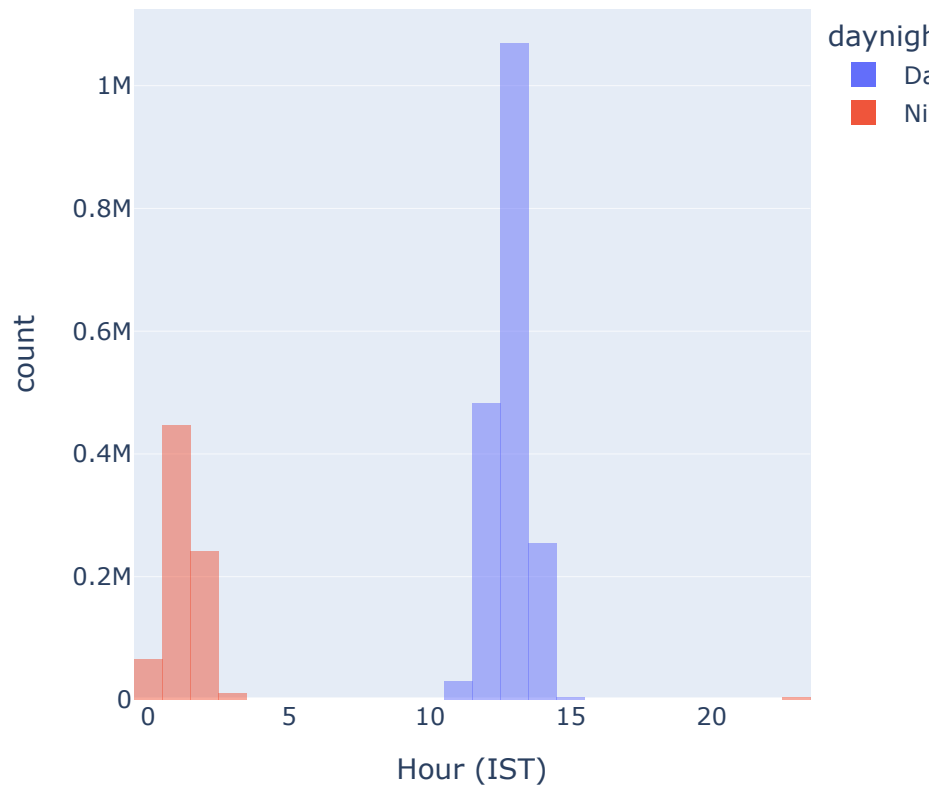
```

Daily Thermal Anomaly Counts



Date

Satellite Pass Times (IST)



Phase 6: Spatial EDA

```
In [11]: # 6.1 Overall spatial distribution – lightweight point layer

# Smaller sample for readability.
map_sample = df.sample(
    n=min(20000, len(df)),
    random_state=42
).copy()

fig = px.scatter_mapbox(
    map_sample,
    lat="latitude",
    lon="longitude",
    color="frp",
    size_max=6,
    opacity=0.35,
    zoom=4,
    mapbox_style="open-street-map",
    hover_data={
        "latitude": ":.4f",
        "longitude": ":.4f",
        "frp": ":.2f",
        "bright_ti4": ":.2f",
        "confidence": True,
        "satellite_name": True,
        "acq_date": True,
    },
    title="Spatial Distribution of VIIRS Thermal Anomalies"
)

fig.update_traces(marker=dict(size=4))

fig.update_layout(
    height=700,
    margin=dict(l=0, r=0, t=50, b=0)
)

fig.show()

# 6.2 High-FRP hotspot analysis

# Use the top 2% FRP detections.
frp_threshold = df["frp"].quantile(0.98)

high_frp = df[df["frp"] >= frp_threshold].copy()

print(f"98th percentile FRP threshold: {frp_threshold:.2f}")
print(f"High-FRP detections: {len(high_frp):,}")

fig_hot = px.scatter_mapbox(
```

```

high_frp,
lat="latitude",
lon="longitude",
color="frp",
size="frp",
size_max=12,
opacity=0.65,
zoom=4,
mapbox_style="open-street-map",
hover_data={
    "latitude": "::.4f",
    "longitude": "::.4f",
    "frp": "::.2f",
    "bright_ti4": "::.2f",
    "confidence": True,
    "satellite_name": True,
    "acq_date": True,
},
title="High-FRP Spatial Hotspots (Top 2%)"
)

fig_hot.update_layout(
    height=700,
    margin=dict(l=0, r=0, t=50, b=0)
)

fig_hot.show()
# 6.3 Spatial density map

fig_density = px.density_mapbox(
    map_sample,
    lat="latitude",
    lon="longitude",
    z="frp",
    radius=12,
    zoom=4,
    mapbox_style="open-street-map",
    title="Spatial Density of Thermal Anomalies"
)

fig_density.update_layout(
    height=700,
    margin=dict(l=0, r=0, t=50, b=0)
)

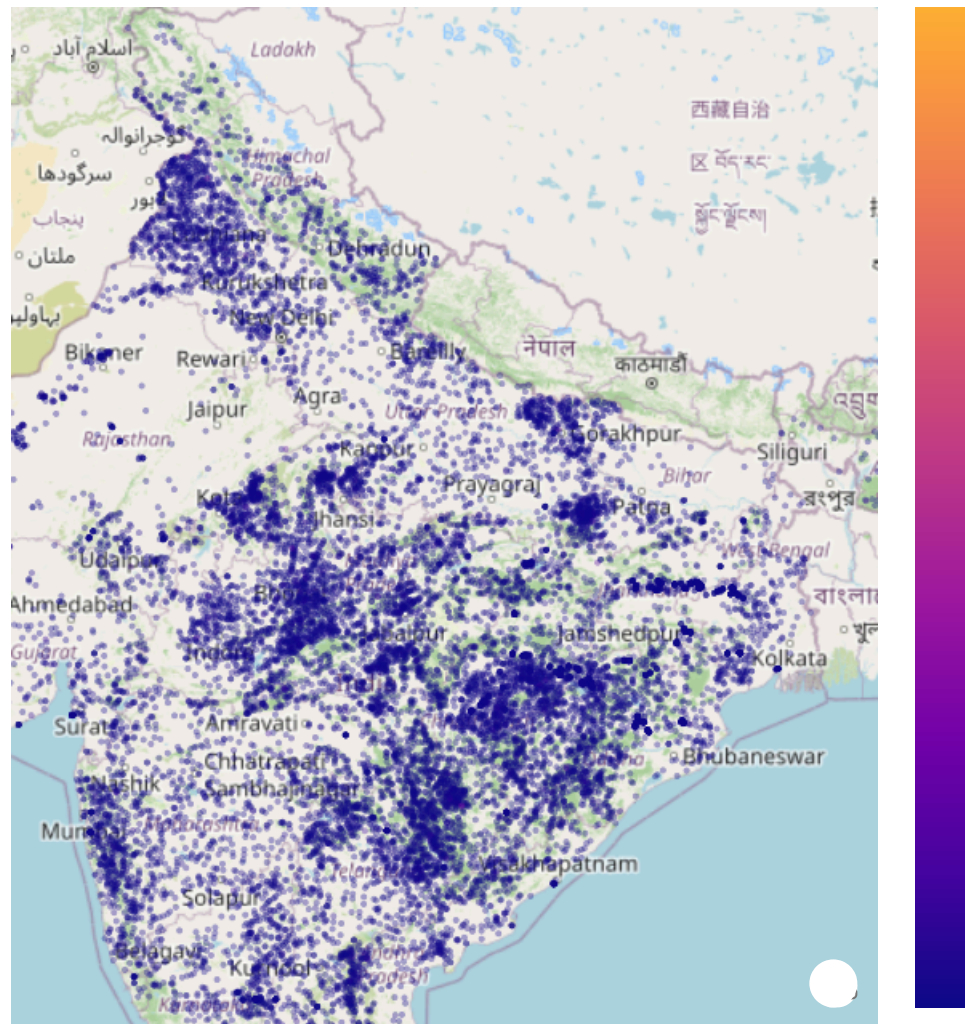
fig_density.show()

```

Spatial Distribution of VIIRS Thermal Anomalies

frp





98th percentile FRP threshold: 28.73
High-FRP detections: 51,967

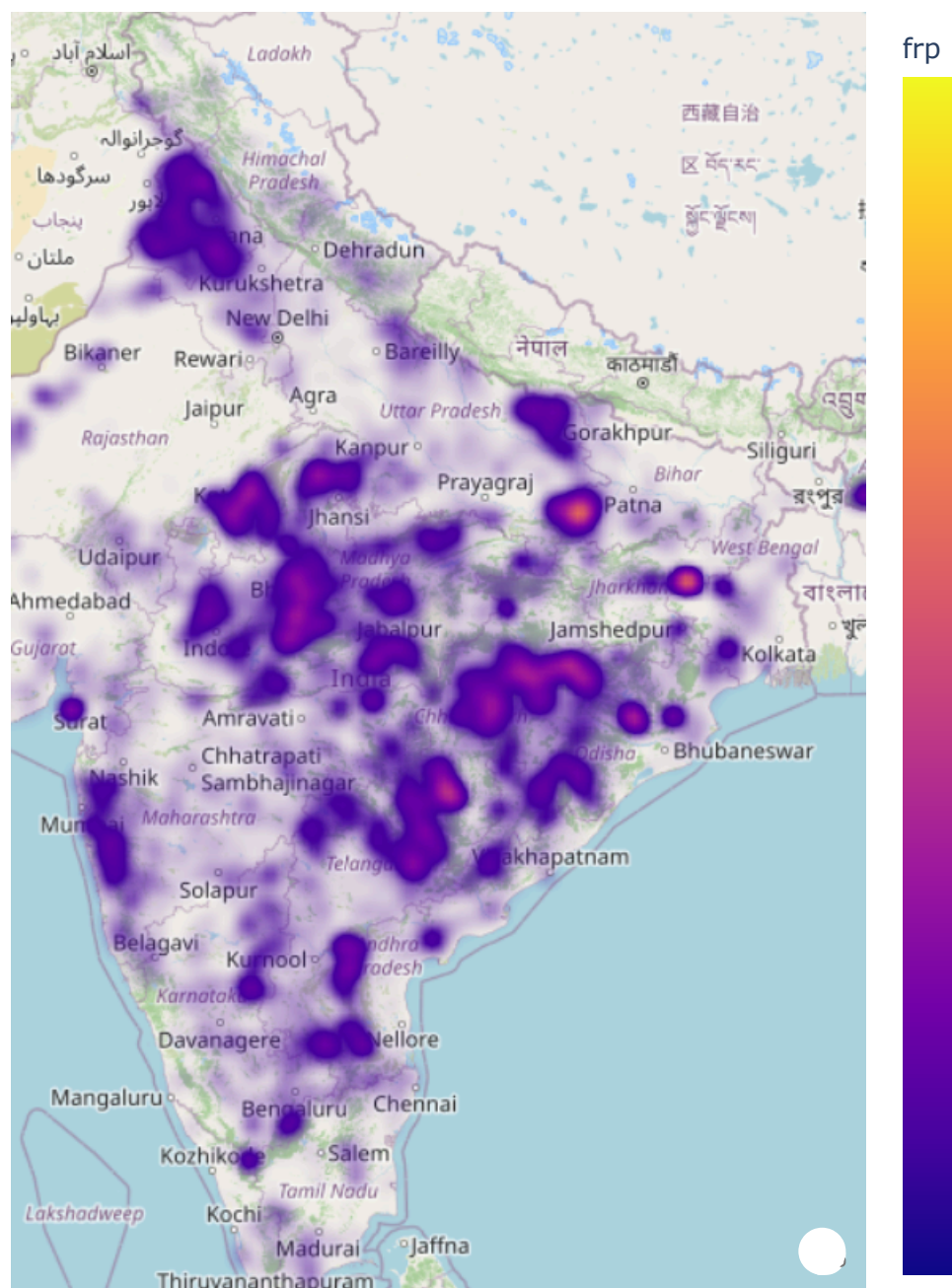
High-FRP Spatial Hotspots (Top 2%)

frp





Spatial Density of Thermal Anomalies



H3 SPATIAL AGGREGATION TO DAILY GRID

```
In [12]: # H3 resolution 8 is used as the daily spatial modeling unit.
# Each VIIRS detection is assigned to an H3 cell and then
# aggregated by H3 cell + acquisition date.

# -----
# 1. Assign every detection to an H3 resolution-8 cell
# -----

# List comprehension is substantially faster than
# df.apply(axis=1) for large datasets.
df["h3_08"] = [
    h3.latlng_to_cell(lat, lon, H3_RES)
    for lat, lon in zip(
        df["latitude"],
        df["longitude"]
    )
]

# Validate H3 assignment before continuing.
assert df["h3_08"].notna().all(), (
    "H3 assignment produced missing values"
)

print(f"Unique H3 cells: {df['h3_08'].nunique():,}")

# -----
# 2. Flag VIIRS I4 saturation
# -----

df["is_saturated"] = (
    df["bright_ti4"] >= (
        SATURATION_TEMP - SATURATION_BUFFER
    )
).astype(int)

# -----
# 3. Aggregate detections to H3 cell + acquisition date
# -----

df_daily = (
    df
    .groupby(
        ["h3_08", "acq_date"],
        as_index=False
    )
    .agg(
        # Fire intensity
        frp_max=("frp", "max"),
```

```

frp_mean=("frp", "mean"),
n_detections=("frp", "count"),

# Thermal characteristics
ti4_max=("bright_ti4", "max"),
is_saturated_max=("is_saturated", "max"),

# VIIRS observation geometry
scan_mean=("scan", "mean"),
track_mean=("track", "mean"),

# Confidence
confidence_high_any=(
    "confidence",
    lambda x: (x == "high").any()
),

# Day/night
daynight=(
    "daynight",
    lambda x: (
        "Both"
        if len(set(x)) > 1
        else x.iloc[0]
    )
),

# Number of unique observing satellites
satellite_nunique=(
    "satellite",
    "nunique"
),

# Archive-derived fire-type flags
# -1 = unknown / NRT
# 0 = false
# 1 = true
is_static_land=(
    "is_static_land",
    "max"
),

is_offshore=(
    "is_offshore",
    "max"
),
)

# -----
# 4. Sort immediately for downstream temporal operations
# -----

df_daily = (
    df_daily
    .sort_values(
        ["h3_08", "acq_date"]

```

```
)  
.reset_index(drop=True)  
)  
  
print(  
    f"Unique H3-days aggregated: "  
    f"{len(df_daily):,}"  
)  
  
print(  
    f"Unique H3 cells in daily table: "  
    f"{df_daily['h3_08'].nunique():,}"  
)  
  
display(df_daily.head())
```

Unique H3 cells: 921,202
Unique H3-days aggregated: 1,718,002
Unique H3 cells in daily table: 921,202

	h3_08	acq_date	frp_max	frp_mean	n_detections	ti4
0	88209a2011ffff	2025-01-26	5.22	5.22	1	33
1	88209a2087ffff	2025-12-15	8.42	8.42	1	34
2	88209a208dffff	2024-10-10	0.87	0.87	1	30
3	88209a20a7ffff	2024-11-06	5.17	5.17	1	32
4	88209a20a7ffff	2024-11-07	3.83	2.54	2	32

```

In [13]: # Collapse the H3-day table across the complete observation
# period to understand which cells repeatedly contain
# thermal anomalies.

h3_spatial = (
    df_daily
    .groupby(
        "h3_08",
        as_index=False
    )
    .agg(
        # Total number of VIIRS detections in the cell
        total_detections=(
            "n_detections",
            "sum"
        ),

        # Number of distinct days on which the cell was active
        active_days=(
            "acq_date",
            "nunique"
        ),

        # Strongest observed FRP in the cell
        max_frp=(
            "frp_max",
            "max"
        ),

        # Mean daily FRP across active days
        mean_frp=(
            "frp_mean",
            "mean"
        ),
    )
)

print(
    f"Unique H3 cells in spatial summary: "
    f"{len(h3_spatial):,}"
)

print("\n--- Top 20 H3 cells by total detections ---")

display(
    h3_spatial
    .sort_values(
        "total_detections",
        ascending=False
    )
    .head(20)
)

print("\n--- Top 20 H3 cells by active days ---")

display(
    h3_spatial

```

```

        .sort_values(
            "active_days",
            ascending=False
        )
        .head(20)
    )

print("\n--- Top 20 H3 cells by maximum FRP ---")

display(
    h3_spatial
        .sort_values(
            "max_frp",
            ascending=False
        )
        .head(20)
    )

```

Unique H3 cells in spatial summary: 921,202

--- Top 20 H3 cells by total detections ---

	h3_08	total_detections	active_days	max_frp	me
190494	883ca9c95bffff	4392	577	14.79	3.2
62783	883c817b29ffff	3863	557	18.36	3.3
97918	883c85d9c3ffff	3727	553	52.27	5.4
158139	883ca04021ffff	3647	572	19.26	3.4
185980	883ca826e9ffff	3432	554	13.53	2.9
629495	8842d9d2e5ffff	3134	601	41.76	5.9
112930	883c8d8ae9ffff	2713	565	12.51	3.2
77710	883c83b5d3ffff	2666	545	20.52	4.1
629496	8842d9d2e7ffff	2446	596	47.93	7.2
93099	883c854a19ffff	2421	529	11.84	2.7
116394	883c8e8ee7ffff	2377	513	47.05	3.9
62785	883c817b2dffff	2315	510	15.69	2.6
190540	883ca9cb07ffff	2292	548	11.07	2.7
190680	883ca9d9a7ffff	2260	543	11.81	2.9
97922	883c85d9cbffff	2247	511	42.79	4.8
196585	883cab6ce5ffff	2201	512	8.53	1.9
782059	8860a1b883ffff	2097	503	11.43	2.5
62780	883c817b23ffff	2060	449	14.74	3.3

	h3_08	total_detections	active_days	max_frp	me
158145	883ca0402dffff	2049	492	15.10	2.9
186183	883ca83661ffff	2006	527	14.29	3.0

--- Top 20 H3 cells by active days ---

	h3_08	total_detections	active_days	max_frp	me
629495	8842d9d2e5ffff	3134	601	41.76	5.9
629496	8842d9d2e7ffff	2446	596	47.93	7.2
190494	883ca9c95bffff	4392	577	14.79	3.2
158139	883ca04021ffff	3647	572	19.26	3.4
112930	883c8d8ae9ffff	2713	565	12.51	3.2
62783	883c817b29ffff	3863	557	18.36	3.3
185980	883ca826e9ffff	3432	554	13.53	2.9
629494	8842d9d2e3ffff	1454	553	56.34	6.5
97918	883c85d9c3ffff	3727	553	52.27	5.4
190540	883ca9cb07ffff	2292	548	11.07	2.7
77710	883c83b5d3ffff	2666	545	20.52	4.1
190680	883ca9d9a7ffff	2260	543	11.81	2.9
112914	883c8d8a85ffff	1873	534	12.85	2.7
87461	883c84a909ffff	1917	533	13.02	3.4
190475	883ca9c86dffff	1974	532	16.82	3.9
190512	883ca9c9c5ffff	1794	530	15.71	2.8
93099	883c854a19ffff	2421	529	11.84	2.7
186183	883ca83661ffff	2006	527	14.29	3.0
186188	883ca8366bffff	1431	526	19.52	3.1
111583	883c8d2ec5ffff	1967	525	13.37	2.4

--- Top 20 H3 cells by maximum FRP ---

	h3_08	total_detections	active_days	max_frp	m
268934	883cc1598bffff	16	2	1074.64	146

	h3_08	total_detections	active_days	max_frp	m
304963	883cea6883fffff	3	1	906.67	313
268994	883cc15a8bfffff	1	1	890.08	890
303909	883cea4a8bfffff	11	1	868.25	149
53314	883c8054cbfffff	1	1	820.37	820
273128	883cc34aa5fffff	4	2	810.83	142
273130	883cc34aadfffff	3	1	810.83	285
264396	883cc072cbfffff	10	3	806.57	146
269098	883cc18257fffff	4	2	801.17	255
273196	883cc34c67fffff	13	2	801.01	138
302956	883cea2e8dfffff	6	1	751.50	193
273282	883cc34f1dfffff	8	1	717.23	295
273280	883cc34f19fffff	10	1	717.23	272
281394	883cccd123fffff	10	2	713.98	136
268372	883cc14725fffff	8	1	713.70	167
279953	883cc56c99fffff	5	1	696.49	303
279958	883cc56cd3fffff	5	2	696.49	312
265290	883cc08c95fffff	2	2	685.38	388
269720	883cc199b3fffff	4	1	655.38	330
269378	883cc18b59fffff	2	1	655.38	330

```

In [14]: # Do NOT attempt to render all ~900k H3 cells.
# Plot only the most active cells so the map remains responsive
# and visually interpretable.

TOP_H3 = 5000

h3_plot = (
    h3_spatial
    .sort_values(
        "total_detections",
        ascending=False
    )
    .head(TOP_H3)
    .copy()
)

print(
    f"Rendering {len(h3_plot):,} most active H3 cells "
    f"out of {len(h3_spatial):,} total cells."
)

# -----
# Convert H3 cells to GeoJSON polygons
# -----

def h3_polygon(cell):
    # h3.cell_to_boundary() returns:
    # [(lat, lon), ...]
    #
    # GeoJSON expects:
    # [(lon, lat), ...]

    boundary = h3.cell_to_boundary(cell)

    return [
        [lon, lat]
        for lat, lon in boundary
    ]

# List comprehension avoids unnecessary pandas apply overhead.
h3_plot["polygon"] = [
    h3_polygon(cell)
    for cell in h3_plot["h3_08"]
]

features = [
    {
        "type": "Feature",
        "id": row["h3_08"],
        "geometry": {
            "type": "Polygon",
            "coordinates": [row["polygon"]],
        },
        "properties": {
            "h3_08": row["h3_08"],
            "total_detections": int(

```

```

        row["total_detections"]
    ),
    "active_days": int(
        row["active_days"]
    ),
    "max_frp": float(
        row["max_frp"]
    ),
},
}
for _, row in h3_plot.iterrows()
]

h3_geojson = {
    "type": "FeatureCollection",
    "features": features,
}

# -----
# Plot
# -----

fig_h3 = px.choropleth_mapbox(
    h3_plot,
    geojson=h3_geojson,
    locations="h3_08",
    featureidkey="id",

    # Color = how often this spatial cell was active
    color="total_detections",

    hover_name="h3_08",

    hover_data={
        "total_detections": True,
        "active_days": True,
        "max_frp": ":.2f",
        "h3_08": False,
    },

    mapbox_style="open-street-map",

    # South Asia view
    zoom=4,

    title=(
        f"H3 Resolution-{H3_RES} "
        f"Detection Density – Top {TOP_H3} Cells"
    ),
)

fig_h3.update_layout(
    height=750,
    margin=dict(
        l=0,
        r=0,
        t=55,

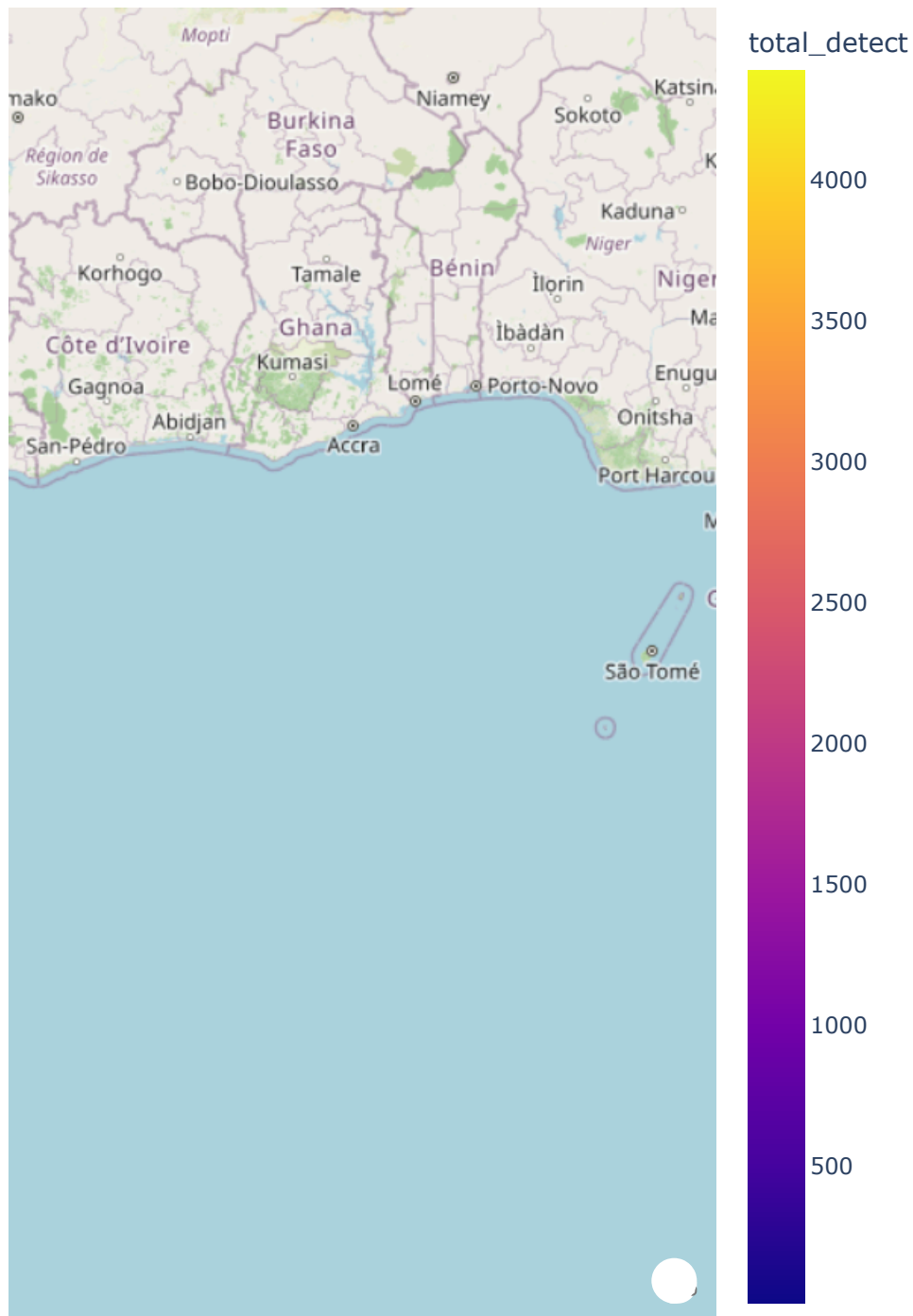
```

```
        b=0
    ),
)

fig_h3.show()
```

Rendering 5,000 most active H3 cells out of 921,202 total cells.

H3 Resolution-8 Detection Density — Top 5000 Cells



```

In [15]: # Detection frequency and thermal intensity represent
# different spatial phenomena.
#
# This map highlights cells containing the strongest
# observed thermal anomalies.

h3_frp_plot = (
    h3_spatial
    .sort_values(
        "max_frp",
        ascending=False
    )
    .head(TOP_H3)
    .copy()
)

print(
    f"Rendering {len(h3_frp_plot):,} "
    f"highest-FRP H3 cells."
)

# -----
# Convert H3 cells to polygons
# -----

h3_frp_plot["polygon"] = [
    h3_polygon(cell)
    for cell in h3_frp_plot["h3_08"]
]

features_frp = [
    {
        "type": "Feature",
        "id": row["h3_08"],
        "geometry": {
            "type": "Polygon",
            "coordinates": [row["polygon"]],
        },
        "properties": {
            "h3_08": row["h3_08"],
            "total_detections": int(
                row["total_detections"]
            ),
            "active_days": int(
                row["active_days"]
            ),
            "max_frp": float(
                row["max_frp"]
            ),
        },
    }
    for _, row in h3_frp_plot.iterrows()
]

h3_geojson_frp = {
    "type": "FeatureCollection",
    "features": features_frp,
}

```

```

}

# -----
# Plot
# -----

fig_h3_frp = px.choropleth_mapbox(
    h3_frp_plot,
    geojson=h3_geojson_frp,
    locations="h3_08",
    featureidkey="id",

    # Color = maximum observed FRP
    color="max_frp",

    hover_name="h3_08",

    hover_data={
        "max_frp": ":.2f",
        "total_detections": True,
        "active_days": True,
        "h3_08": False,
    },

    mapbox_style="open-street-map",

    zoom=4,

    title=(
        f"H3 Resolution-{H3_RES} "
        f"Highest-FRP Hotspots – Top {TOP_H3} Cells"
    ),
)

fig_h3_frp.update_layout(
    height=750,
    margin=dict(
        l=0,
        r=0,
        t=55,
        b=0
    ),
)

fig_h3_frp.show()

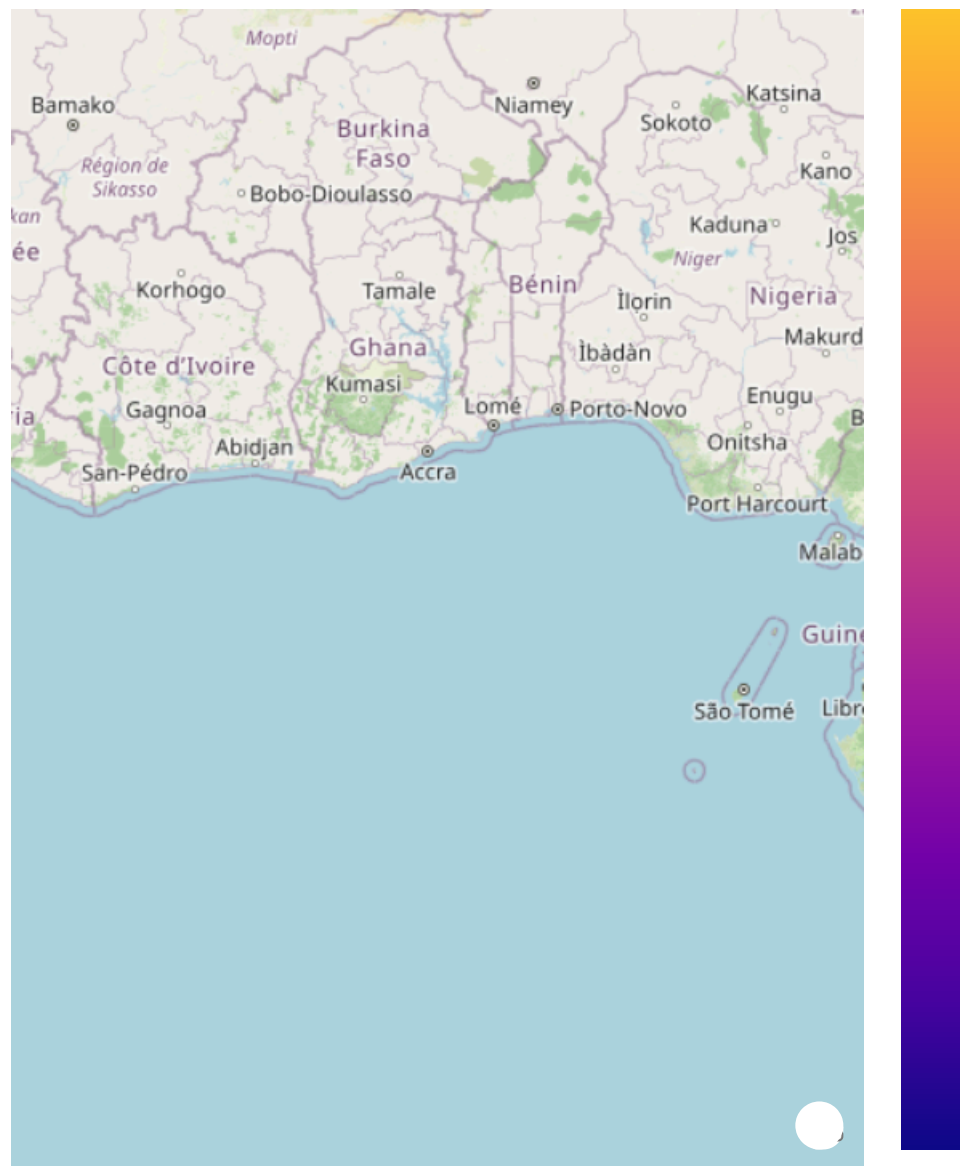
```

Rendering 5,000 highest-FRP H3 cells.

H3 Resolution-8 Highest-FRP Hotspots — Top 5000 C

max_





CALENDAR-AWARE LEAKAGE-SAFE ROLLING FEATURES

```
In [16]: # =====
# PHASE 8: FAST CALENDAR-AWARE LEAKAGE-SAFE ROLLING FEATURES
# =====

# Purpose:
# Add historical context to every H3-day row.
#
# Historical features describe what happened in the SAME H3
# cell BEFORE the current acquisition date.
#
# IMPORTANT:
# shift(1) happens BEFORE rolling.
# Therefore today's observation cannot leak into today's
# historical features.
#
# This implementation DOES NOT use groupby.apply().
# That is critical because this dataset has ~921k H3 cells.

import gc

print("=" * 70)
print("PHASE 8 – FAST ROLLING FEATURES")
print("=" * 70)

# -----
# 1. Sort once
# -----

df_daily = (
    df_daily
    .sort_values(
        ["h3_08", "acq_date"]
    )
    .reset_index(drop=True)
)

print(f"\nRows: {len(df_daily):,}")
print(f"Unique H3 cells: {df_daily['h3_08'].nunique():,}")
print(
    f"Date range: "
    f"{df_daily['acq_date'].min()} → "
    f"{df_daily['acq_date'].max()}"
)

# -----
# 2. Create leakage-safe previous-day features
# -----
```

```

# Previous FRP within the SAME H3 cell.
#
# First observation of every H3 cell becomes NaN.

df_daily["frp_prev"] = (
    df_daily
    .groupby(
        "h3_08",
        sort=False
    )["frp_max"]
    .shift(1)
)

# Every row in df_daily represents one active H3-day.
#
# Therefore:
#   first row of a cell = 0 previous active days
#   subsequent rows    = 1 previous active day
#
# This is calculated before rolling.

df_daily["active_prev"] = (
    df_daily
    .groupby(
        "h3_08",
        sort=False
    )
    .cumcount()
    .gt(0)
    .astype("int8")
)

print("\nPrevious-observation columns created.")

# -----
# 3. Temporary rolling input
# -----

rolling_input = df_daily[
    [
        "h3_08",
        "acq_date",
        "frp_prev",
        "active_prev",
    ]
].copy()

# Keep acquisition date as the time column.
# The H3 cell remains the grouping column.

# =====
# 4. 7-DAY HISTORICAL WINDOW
# =====

print("\nCalculating 7-day historical features...")

rolling_7d = (

```

```

rolling_input
.groupby(
    "h3_08",
    sort=False
)
.rolling(
    "7D",
    on="acq_date",
    min_periods=1
)[
    [
        "frp_prev",
        "active_prev",
    ]
]
.agg({
    "frp_prev": "max",
    "active_prev": "sum",
})
.reset_index()
)

rolling_7d = rolling_7d.rename(
    columns={
        "frp_prev": "frp_max_lag7",
        "active_prev": "active_days_7d",
    }
)

# -----
# Verify exact row alignment before assigning values
# -----

assert rolling_7d[
    ["h3_08", "acq_date"]
].reset_index(drop=True).equals(
    df_daily[
        ["h3_08", "acq_date"]
    ].reset_index(drop=True)
), "7D rolling output is not aligned with df_daily."

df_daily["frp_max_lag7"] = (
    rolling_7d["frp_max_lag7"].to_numpy()
)

df_daily["active_days_7d"] = (
    rolling_7d["active_days_7d"].to_numpy()
)

del rolling_7d
gc.collect()

print("7-day features complete.")

# =====
# 5. 30-DAY HISTORICAL WINDOW
# =====

```

```

print("\nCalculating 30-day historical features...")

rolling_30d = (
    rolling_input
    .groupby(
        "h3_08",
        sort=False
    )
    .rolling(
        "30D",
        on="acq_date",
        min_periods=1
    )[
        [
            "frp_prev",
            "active_prev",
        ]
    ]
    .agg({
        "frp_prev": "max",
        "active_prev": "sum",
    })
    .reset_index()
)

rolling_30d = rolling_30d.rename(
    columns={
        "frp_prev": "frp_max_lag30",
        "active_prev": "active_days_30d",
    }
)

# -----
# Verify alignment
# -----

assert rolling_30d[
    ["h3_08", "acq_date"]
].reset_index(drop=True).equals(
    df_daily[
        ["h3_08", "acq_date"]
    ].reset_index(drop=True)
), "30D rolling output is not aligned with df_daily."

df_daily["frp_max_lag30"] = (
    rolling_30d["frp_max_lag30"].to_numpy()
)

df_daily["active_days_30d"] = (
    rolling_30d["active_days_30d"].to_numpy()
)

del rolling_30d
gc.collect()

print("30-day features complete.")

```

```

# =====
# 6. 90-DAY HISTORICAL WINDOW
# =====

print("\nCalculating 90-day historical features...")

rolling_90d = (
    rolling_input
    .groupby(
        "h3_08",
        sort=False
    )
    .rolling(
        "90D",
        on="acq_date",
        min_periods=1
    )
    [
        "frp_prev",
        "active_prev",
    ]
    .agg({
        "frp_prev": "max",
        "active_prev": "sum",
    })
    .reset_index()
)

rolling_90d = rolling_90d.rename(
    columns={
        "active_prev": "active_days_90d",
    }
)

# We only need active_days_90d from the 90-day pass.
# There is intentionally NO frp_max_lag90 feature.

# -----
# Verify alignment
# -----

assert rolling_90d[
    ["h3_08", "acq_date"]
].reset_index(drop=True).equals(
    df_daily[
        ["h3_08", "acq_date"]
    ].reset_index(drop=True)
), "90D rolling output is not aligned with df_daily."

df_daily["active_days_90d"] = (
    rolling_90d["active_days_90d"].to_numpy()
)

del rolling_90d
del rolling_input

```

```

gc.collect()

print("90-day features complete.")

# -----
# 7. Temporary columns are no longer needed
# -----

df_daily.drop(
    columns=[
        "frp_prev",
        "active_prev",
    ],
    inplace=True,
)

# -----
# 8. Restore final ordering
# -----

df_daily = (
    df_daily
    .sort_values(
        ["h3_08", "acq_date"]
    )
    .reset_index(drop=True)
)

print("\n" + "=" * 70)
print("PHASE 8 COMPLETE")
print("=" * 70)

print(
    f"\nRows: {len(df_daily):,}"
)

print(
    f"Unique H3 cells: "
    f"{df_daily['h3_08'].nunique():,}"
)

print("\nCreated features:")
print("  frp_max_lag7")
print("  frp_max_lag30")
print("  active_days_7d")
print("  active_days_30d")
print("  active_days_90d")

display(
    df_daily[
        [
            "h3_08",
            "acq_date",
            "frp_max",
            "frp_max_lag7",
            "frp_max_lag30",
            "active_days_7d",

```

```
        "active_days_30d",
        "active_days_90d",
    ]
].head(20)
)
```

```
=====
=====
```

PHASE 8 – FAST ROLLING FEATURES

```
=====
=====
```

Rows: 1,718,002

Unique H3 cells: 921,202

Date range: 2024-08-01 00:00:00 → 2026-08-01 00:00:00

Previous-observation columns created.

Calculating 7-day historical features...

7-day features complete.

Calculating 30-day historical features...

30-day features complete.

Calculating 90-day historical features...

90-day features complete.

```
=====
=====
PHASE 8 COMPLETE
=====
=====
```

Rows: 1,718,002

Unique H3 cells: 921,202

Created features:

```
frp_max_lag7
frp_max_lag30
active_days_7d
active_days_30d
active_days_90d
```

	h3_08	acq_date	frp_max	frp_max_lag7	frp_max_lag30
0	88209a2011fffff	2025-01-26	5.22	NaN	NaN
1	88209a2087fffff	2025-12-15	8.42	NaN	NaN
2	88209a208dfffff	2024-10-10	0.87	NaN	NaN
3	88209a20a7fffff	2024-11-06	5.17	NaN	NaN

	h3_08	acq_date	frp_max	frp_max_lag7	frp_max_lag30
4	88209a20a7ffff	2024-11-07	3.83	5.17	5.17
5	88209a20a9ffff	2024-12-30	1.04	NaN	NaN
6	88209a20abffff	2025-12-14	2.85	NaN	NaN
7	88209a20abffff	2025-12-15	16.22	2.85	2.85
8	88209a20b3ffff	2025-11-26	1.24	NaN	NaN
9	88209a20b9ffff	2025-12-16	14.92	NaN	NaN
10	88209a20bbffff	2025-12-16	10.66	NaN	NaN
11	88209a20bdffff	2025-12-15	20.65	NaN	NaN
12	88209a20bdffff	2025-12-16	8.36	20.65	20.65
13	88209a21d3ffff	2024-10-27	4.82	NaN	NaN
14	88209a2291ffff	2025-11-29	17.43	NaN	NaN
15	88209a2297ffff	2025-03-23	3.74	NaN	NaN
16	88209a2297ffff	2025-11-29	8.08	3.74	3.74
17	88209a2297ffff	2025-12-02	5.54	8.08	8.08
18	88209a252dffff	2025-11-28	5.09	NaN	NaN
19	88209a2567ffff	2025-11-29	5.41	NaN	NaN

In [17]:

```

# =====
# PHASE 8A: ROLLING FEATURE VALIDATION
# =====

rolling_cols = [
    "frp_max_lag7",
    "frp_max_lag30",
    "active_days_7d",
    "active_days_30d",
    "active_days_90d",
]

print("=" * 70)
print("PHASE 8A – ROLLING FEATURE VALIDATION")
print("=" * 70)

# -----
# 1. Missing-value check
# -----

print("\n--- Missing values ---")

display(
    df_daily[rolling_cols]
    .isna()
    .sum()
)

# FRP lag features can legitimately have NaN for an H3 cell's
# first observed day.
#
# Active-day features should contain numeric zeros/values.

assert (
    df_daily[
        [
            "active_days_7d",
            "active_days_30d",
            "active_days_90d",
        ]
    ]
    .isna()
    .sum()
    .sum()
) == 0, (
    "ERROR: active-day rolling features contain unexpected NaN."
)

# -----
# 2. Range checks
# -----

print("\n--- Active-day ranges ---")

print(
    "7D :",
    df_daily["active_days_7d"].min(),

```

```

        "→",
        df_daily["active_days_7d"].max()
    )

    print(
        "30D:",
        df_daily["active_days_30d"].min(),
        "→",
        df_daily["active_days_30d"].max()
    )

    print(
        "90D:",
        df_daily["active_days_90d"].min(),
        "→",
        df_daily["active_days_90d"].max()
    )

    assert df_daily["active_days_7d"].between(
        0, 7
    ).all(), "ERROR: active_days_7d outside 0-7."

    assert df_daily["active_days_30d"].between(
        0, 30
    ).all(), "ERROR: active_days_30d outside 0-30."

    assert df_daily["active_days_90d"].between(
        0, 90
    ).all(), "ERROR: active_days_90d outside 0-90."

    print("\nActive-day range checks passed.")

    # -----
    # 3. Feature summary
    # -----

    print("\n--- Rolling feature summary ---")

    display(
        df_daily[rolling_cols].describe(
            percentiles=[
                0.50,
                0.75,
                0.90,
                0.95,
                0.99,
            ]
        )
    )

```

```

=====
====
PHASE 8A — ROLLING FEATURE VALIDATION
=====
====

--- Missing values ---

```

```
frp_max_lag7      921202
frp_max_lag30     921202
active_days_7d      0
active_days_30d     0
active_days_90d     0
dtype: int64
```

--- Active-day ranges ---

7D : 0.0 → 7.0

30D: 0.0 → 30.0

90D: 0.0 → 90.0

Active-day range checks passed.

--- Rolling feature summary ---

	frp_max_lag7	frp_max_lag30	active_days_7d	active_days_30d
count	796800.000000	796800.000000	1.718002e+06	1.718002e+06
mean	6.144644	6.690611	8.808593e-01	2.185428e+00
std	12.647896	13.261621	1.490189e+00	5.610345e+00
min	0.000000	0.000000	0.000000e+00	0.000000e+00
50%	3.740000	4.110000	0.000000e+00	0.000000e+00
75%	6.540000	7.130000	1.000000e+00	1.000000e+00
90%	11.160000	12.060000	2.000000e+00	4.000000e+00
95%	16.420000	17.880000	5.000000e+00	1.700000e+01
99%	44.320100	47.810000	7.000000e+00	2.800000e+01
max	810.830000	810.830000	7.000000e+00	3.000000e+01



In [18]:

```

# =====
# PHASE 8B: EXPLICIT TEMPORAL LEAKAGE CHECK
# =====

print("=" * 70)
print("PHASE 8B – TEMPORAL LEAKAGE CHECK")
print("=" * 70)

# The first observed day of every H3 cell has no history.
#
# Therefore:
#   frp_max_lag7      = NaN
#   frp_max_lag30     = NaN
#   active_days_7d    = 0
#   active_days_30d   = 0
#   active_days_90d   = 0

first_rows = (
    df_daily
    .sort_values(
        ["h3_08", "acq_date"]
    )
    .groupby(
        "h3_08",
        sort=False
    )
    .head(1)
)

print(
    f"First observed H3-days checked: "
    f"{len(first_rows):,}"
)

# First row cannot have previous FRP.
assert first_rows[
    "frp_max_lag7"
].isna().all(), (
    "ERROR: first H3-day has a 7D historical FRP value."
)

assert first_rows[
    "frp_max_lag30"
].isna().all(), (
    "ERROR: first H3-day has a 30D historical FRP value."
)

# First row cannot have previous active days.
assert (
    first_rows["active_days_7d"] == 0
).all(), (
    "ERROR: first H3-day has previous 7D activity."
)

assert (
    first_rows["active_days_30d"] == 0
).all(), (

```

```

        "ERROR: first H3-day has previous 30D activity."
    )

    assert (
        first_rows["active_days_90d"] == 0
    ).all(), (
        "ERROR: first H3-day has previous 90D activity."
    )

    print("\nFirst-day historical-feature check passed.")

    # -----
    # Verify chronological order
    # -----

    is_sorted = (
        df_daily
        .groupby("h3_08")["acq_date"]
        .apply(lambda x: x.is_monotonic_increasing)
        .all()
    )

    assert is_sorted, (
        "ERROR: H3 dates are not monotonically increasing."
    )

    print("H3 chronological-order check passed.")

    print("\n" + "=" * 70)
    print("PHASE 8 LEAKAGE CHECK PASSED")
    print("=" * 70)

```

```

=====
====
PHASE 8B – TEMPORAL LEAKAGE CHECK
=====
====
First observed H3-days checked: 921,202

First-day historical-feature check passed.
H3 chronological-order check passed.

=====
====
PHASE 8 LEAKAGE CHECK PASSED
=====
====

```

FEATURE FINALIZATION + HARD VALIDATION

```
In [19]: # =====
# PHASE 9: FEATURE FINALIZATION
# =====

# Purpose:
# Convert Phase 8's raw rolling output into a model-ready
# table for CatBoost: explicit missingness flags instead of
# lossy fills, dtype downcasting, and a clean feature manifest.

print("=" * 70)
print("PHASE 9 – FEATURE FINALIZATION")
print("=" * 70)

print(f"\nRows before finalization: {len(df_daily):,}")
print(f"Memory before: {df_daily.memory_usage(deep=True).sum() /
1e6:.2f} MB")

# -----
# 1. Explicit missingness flag for lag features
# -----

# frp_max_lag7 and frp_max_lag30 share the identical NaN mask
# (both are NaN iff this is the H3 cell's first-ever observed
# active day). One flag covers both.

df_daily["is_first_observation"] = (
    df_daily["frp_max_lag7"].isna().astype("int8")
)

assert (
    df_daily["is_first_observation"].sum()
    == df_daily["frp_max_lag7"].isna().sum()
    == df_daily["frp_max_lag30"].isna().sum()
), "frp_max_lag7 / frp_max_lag30 NaN masks diverge – investigate before
finalizing."

print(
    f"\nFirst-observation rows flagged: "
    f"{df_daily['is_first_observation'].sum():,}"
)

# NaNs in frp_max_lag7 / frp_max_lag30 are kept intentionally –
# CatBoost handles missing numerics natively. Do NOT fillna(0)
# here; see notebook notes on why that collides with real FRP=0.

# -----
# 2. Target labeling clarity
# -----
```

```

# is_static_land: -1 = unlabeled (no archive-derived fire_type
# that day), 0 = vegetation-type detection that day, 1 = at
# least one static-land-source (industrial-proxy) detection.

df_daily["is_labeled"] = (
    df_daily["is_static_land"] != -1
).astype("int8")

print(
    f"\nLabeled H3-days: "
    f"{df_daily['is_labeled'].sum():,} "
    f"({df_daily['is_labeled'].mean() * 100:.2f}%)"
)

# -----
# 3. Dtype downcasting
# -----

dtype_map = {
    "h3_08": "category",
    "daynight": "category",
    "frp_max": "float32",
    "frp_mean": "float32",
    "ti4_max": "float32",
    "scan_mean": "float32",
    "track_mean": "float32",
    "frp_max_lag7": "float32",
    "frp_max_lag30": "float32",
    "n_detections": "int16",
    "satellite_nunique": "int8",
    "is_saturated_max": "int8",
    "is_static_land": "int8",
    "is_offshore": "int8",
    "active_days_7d": "int8",
    "active_days_30d": "int8",
    "active_days_90d": "int8",
}

for col, dtype in dtype_map.items():
    df_daily[col] = df_daily[col].astype(dtype)

print("\nDtype downcasting applied.")

print(f"\nMemory after: {df_daily.memory_usage(deep=True).sum() /
1e6:.2f} MB")

# -----
# 4. Feature manifest for downstream CatBoost training
# -----

identifier_cols = ["h3_08", "acq_date"]
target_col = "is_static_land"
label_mask_col = "is_labeled"

categorical_features = ["daynight"]

```

```

numeric_features = [
    "frp_max", "frp_mean", "n_detections", "ti4_max",
    "is_saturated_max", "scan_mean", "track_mean",
    "satellite_nunique", "is_offshore",
    "frp_max_lag7", "frp_max_lag30",
    "active_days_7d", "active_days_30d", "active_days_90d",
    "is_first_observation",
]

boolean_features = ["confidence_high_any"]

print("\n--- Feature manifest ---")
print(f"Identifiers ({len(identifier_cols)}): {identifier_cols}")
print(f"Target: {target_col} | Label mask: {label_mask_col}")
print(f"Categorical ({len(categorical_features)}): {categorical_features}")
print(f"Boolean ({len(boolean_features)}): {boolean_features}")
print(f"Numeric ({len(numeric_features)}): {numeric_features}")

print("\n" + "=" * 70)
print("PHASE 9 COMPLETE")
print("=" * 70)

display(df_daily.head(10))

```

```

=====
====
PHASE 9 – FEATURE FINALIZATION
=====
====

Rows before finalization: 1,718,002
Memory before: 421.78 MB

First-observation rows flagged: 921,202

Labeled H3-days: 1,644,273 (95.71%)

Dtype downcasting applied.

Memory after: 183.83 MB

--- Feature manifest ---
Identifiers (2): ['h3_08', 'acq_date']
Target: is_static_land | Label mask: is_labeled
Categorical (1): ['daynight']
Boolean (1): ['confidence_high_any']
Numeric (15): ['frp_max', 'frp_mean', 'n_detections', 'ti4_max',
               'is_saturated_max', 'scan_mean', 'track_mean',
               'satellite_nunique', 'is_offshore', 'frp_max_lag7',
               'frp_max_lag30', 'active_days_7d', 'active_days_30d',
               'active_days_90d', 'is_first_observation']

=====
====
PHASE 9 COMPLETE

```

=====

=====

	h3_08	acq_date	frp_max	frp_mean	n_detections
0	88209a2011ffff	2025-01-26	5.220000	5.220000	1
1	88209a2087ffff	2025-12-15	8.420000	8.420000	1
2	88209a208dffff	2024-10-10	0.870000	0.870000	1
3	88209a20a7ffff	2024-11-06	5.170000	5.170000	1
4	88209a20a7ffff	2024-11-07	3.830000	2.540000	2
5	88209a20a9ffff	2024-12-30	1.040000	1.040000	1
6	88209a20abffff	2025-12-14	2.850000	2.850000	1
7	88209a20abffff	2025-12-15	16.219999	16.219999	1
8	88209a20b3ffff	2025-11-26	1.240000	0.830000	2
9	88209a20b9ffff	2025-12-16	14.920000	11.880000	4

◀ ◻ ▶

In [20]:

```

# =====
# PHASE 9A: HARD VALIDATION
# =====

print("=" * 70)
print("PHASE 9A – HARD VALIDATION")
print("=" * 70)

# -----
# 1. Row/cell integrity – finalization must not drop anything
# -----

assert len(df_daily) == 1_718_002, (
    f"ERROR: row count changed during finalization "
    f"({len(df_daily):,} != 1,718,002)."
)

assert df_daily["h3_08"].nunique() == 921_202, (
    f"ERROR: H3 cell count changed during finalization "
    f"({df_daily['h3_08'].nunique():,} != 921,202)."
)

print("\nRow/cell integrity check passed.")

# -----
# 2. NaN allowlist – only the two lag columns may be NaN
# -----

allowed_nan_cols = {"frp_max_lag7", "frp_max_lag30"}

nan_counts = df_daily.isna().sum()
unexpected_nan = nan_counts[
    (nan_counts > 0) & (~nan_counts.index.isin(allowed_nan_cols))
]

assert unexpected_nan.empty, (
    f"ERROR: unexpected NaN outside allowlist:\n{unexpected_nan}"
)

print("\nNaN allowlist check passed.")
print("\n--- Remaining NaN (expected) ---")
display(nan_counts[nan_counts > 0])

# -----
# 3. Lag NaN count must equal is_first_observation count
# -----

assert (
    df_daily["frp_max_lag7"].isna().sum()
    == df_daily["is_first_observation"].sum()
), "ERROR: is_first_observation flag drifted from actual NaN count."

print("is_first_observation consistency check passed.")

# -----
# 4. Target label coverage report (not gated – reporting only)
# -----

```

```

print("\n--- is_static_land distribution (target) ---")
display(df_daily["is_static_land"].value_counts(dropna=False))

print(
    f"\nUnlabeled H3-days (is_static_land == -1): "
    f"{(df_daily['is_static_land'] == -1).sum():,} "
    f"({(df_daily['is_static_land'] == -1).mean() * 100:.2f}%) "
)

# -----
# 5. Dtype / memory report
# -----

print("\n--- Final dtypes ---")
display(df_daily.dtypes)

print(f"\nFinal memory usage: {df_daily.memory_usage(deep=True).sum() /
1e6:.2f} MB")

print("\n" + "=" * 70)
print("PHASE 9A VALIDATION PASSED")
print("=" * 70)

```

```

=====
====
PHASE 9A — HARD VALIDATION
=====
=====

```

Row/cell integrity check passed.

NaN allowlist check passed.

--- Remaining NaN (expected) ---

```

frp_max_lag7      921202
frp_max_lag30     921202
dtype: int64

```

is_first_observation consistency check passed.

--- is_static_land distribution (target) ---

```

is_static_land
0      1494838
1      149435
-1      73729
Name: count, dtype: int64

```

Unlabeled H3-days (is_static_land == -1): 73,729 (4.29%)

--- Final dtypes ---

h3_08	category
acq_date	datetime64[ns]
frp_max	float32
frp_mean	float32
n_detections	int16
ti4_max	float32
is_saturated_max	int8
scan_mean	float32
track_mean	float32
confidence_high_any	bool
daynight	category
satellite_nunique	int8
is_static_land	int8
is_offshore	int8
frp_max_lag7	float32
active_days_7d	int8
frp_max_lag30	float32
active_days_30d	int8
active_days_90d	int8
is_first_observation	int8
is_labeled	int8
dtype:	object

Final memory usage: 183.83 MB

```
=====
====
PHASE 9A VALIDATION PASSED
=====
=====
```

In [21]:

```

# =====
# PHASE 9B: SAVE FINALIZED FEATURE TABLE
# =====

print("=" * 70)
print("PHASE 9B – SAVE FINALIZED FEATURE TABLE")
print("=" * 70)

df_daily.to_parquet(OUTPUT_PATH, index=False)

print(f"\nSaved to: {OUTPUT_PATH}")

# Verify round-trip integrity before trusting the saved file.
df_check = pd.read_parquet(OUTPUT_PATH)

assert len(df_check) == len(df_daily), (
    "ERROR: row count mismatch after parquet round-trip."
)

assert list(df_check.columns) == list(df_daily.columns), (
    "ERROR: column order/set mismatch after parquet round-trip."
)

del df_check
gc.collect()

print("Round-trip verification passed.")

print("\n" + "=" * 70)
print("PHASE 9 FULLY COMPLETE – feature table ready for modeling")
print("=" * 70)

```

```

=====
====
PHASE 9B – SAVE FINALIZED FEATURE TABLE
=====
====

Saved to: /kaggle/working/sih2026_h3_daily_features.parquet
Round-trip verification passed.

=====
====
PHASE 9 FULLY COMPLETE – feature table ready for modeling
=====
====

```

In []: